

Учебная карта курса

Embedded C/C++ и робототехника: от «Hello, world» до готового робота

Курс рассчитан примерно на полгода занятий. Всего в карте 211 уроков, объединённых в 14 модулей (0-13). Каждый урок имеет собственный код вида «Модуль X.Y» — впишите этот код в начале нового чата, и по нему будет сгенерирован полноценный урок умеренной глубины проработки темы.

Платформенная привязка: модули 0-2 — чистый C на Ubuntu без железа; модули 3-4 — bare-metal на STM32 Blue Pill; модуль 5 — переход на ESP32/ESP-IDF; модуль 6 — C++ в embedded; модуль 7 — моторы, homing, захват, первая 3D-печать; модули 8-13 — FreeRTOS, сборка робота, сенсорика, телеметрия, ROS 2 и дипломный проект.

В отдельные дни допускается параллельное прохождение нескольких уроков из разных направлений — например, урок по C/C++, урок по электронике и практика пайки/сборки в один день.

Модуль	Тема	Уроков
Модуль 0	Инструментарий и подготовка (Ubuntu, без железа)	8
Модуль 1	Язык C: типы, указатели, память	24
Модуль 2	Make и GDB	9
Модуль 3	Bare-metal STM32 Blue Pill: GPIO и регистры	22
Модуль 4	Таймеры, прерывания, ШИМ	16
Модуль 5	Переезд на ESP32/ESP-IDF: протоколы и первый драйвер датчика	20
Модуль 6	C++ в embedded	15
Модуль 7	Шаговые двигатели, homing, электромагнитный захват, 3D-печать	20
Модуль 8	FreeRTOS	15
Модуль 9	Сборка робота	12
Модуль 10	Сенсорика и одометрия	13
Модуль 11	Телеметрия	10
Модуль 12	ROS 2	17
Модуль 13	Дипломный проект	10
Итого		211

Модуль 0. Инструментарий и подготовка (Ubuntu, без железа)

Платформа: Ubuntu, без железа

0.1	Настройка окружения и этапы компиляции (пройден)
0.2	Препроцессор, компилятор, ассемблер, линковщик — разбор флагов gcc
0.3	Структура проекта: заголовочные файлы, единицы трансляции, include guard
0.4	Git: репозиторий, коммиты, ветки, .gitignore для C-проектов
0.5	Инвентаризация железа: STM32 Blue Pill, ESP32-WROOM-32, датчики, документация
0.6	Чтение даташитов и reference manual: как искать нужную информацию
0.7	Настройка удалённой работы между Ubuntu и Windows-ноутбуками (git remote, SSH)
0.8	Итоговая практика модуля 0: мини-проект из нескольких файлов с заготовкой Makefile

Модуль 1. Язык C: типы, указатели, память

1.1	Примитивные типы данных и их размеры (int, char, float, double, size_t)
1.2	Модификаторы типов и диапазоны значений (signed/unsigned, short/long)
1.3	Операторы и приоритет вычислений в выражениях
1.4	Условные конструкции if/else и switch: подводные камни
1.5	Циклы for/while/do-while и управление ими (break, continue, goto)
1.6	Функции: объявление, определение, передача параметров по значению
1.7	Область видимости переменных: локальные, глобальные, static
1.8	Рекурсия: механизм работы через стек вызовов
1.9	Массивы: объявление, инициализация, доступ по индексу
1.10	Многомерные массивы и их представление в памяти
1.11	Строки как массивы char: обработка, стандартные функции string.h
1.12	Введение в указатели: адрес переменной и разыменование
1.13	Арифметика указателей и связь указателей с массивами
1.14	Указатели на указатели и многоуровневая косвенность
1.15	Указатели и строки: динамическая работа с текстом
1.16	Структуры struct: объявление, вложенность, выравнивание в памяти
1.17	Объединения union и перечисления enum
1.18	typedef и создание собственных именованных типов
1.19	Динамическое выделение памяти: malloc, calloc, realloc, free
1.20	Типичные ошибки динамической памяти: утечки, use-after-free, double free
1.21	Указатели на функции и их применение (колбэки)
1.22	Модификаторы const и volatile: гарантии компилятора
1.23	Битовые операции и работа с флагами через маски
1.24	Модель памяти процесса: стек, куча, сегменты данных и BSS

Модуль 2. Make и GDB

Платформа: *Ubuntu, без железа*

2.1	Makefile: базовый синтаксис, цели, зависимости, команды
2.2	Переменные и автоматические переменные в Makefile (\$@, \$<, \$^)
2.3	Сборка многофайлового проекта: объектные файлы и инкрементальная сборка
2.4	Заголовочные файлы и предотвращение повторного включения
2.5	Статические и динамические библиотеки: создание и подключение
2.6	Процесс линковки: разрешение символов, типичные ошибки линкера
2.7	Отладка в GDB: точки останова, шаг за шагом, просмотр переменных
2.8	GDB продвинутый уровень: память, регистры, стек вызовов
2.9	Valgrind и поиск ошибок работы с памятью

Модуль 3. Bare-metal STM32 Blue Pill: GPIO и регистры

Платформа: *STM32 Blue Pill (bare-metal)*

3.1	Знакомство с STM32 Blue Pill: архитектура ARM Cortex-M3, распиновка
3.2	Reference Manual STM32F103: структура документа, карта памяти
3.3	Memory-mapped I/O: отображение регистров периферии в адресное пространство
3.4	Дерево тактирования (clock tree): HSE, HSI, PLL, системная частота
3.5	Регистр RCC: включение тактирования периферии GPIO
3.6	Регистры GPIO (MODER/CRL-CRH): режимы входа и выхода
3.7	Регистр ODR: мигание светодиодом напрямую через регистры
3.8	Регистр IDR: чтение состояния входа, опрос кнопки
3.9	Настройка подтяжки (pull-up/pull-down) на входах
3.10	Типы выходов: push-pull и open-drain, когда что использовать
3.11	Скорость переключения выхода (OSPEEDR) и её влияние
3.12	ST-Link и загрузка прошивки: st-flash, OpenOCD
3.13	Введение в линкер-скрипт: секции .text, .data, .bss
3.14	Стартовый код и таблица векторов прерываний
3.15	Указатель стека и точка входа Reset_Handler
3.16	SysTick: организация простой программной задержки
3.17	Практика: мигание несколькими светодиодами с разной частотой
3.18	Практика: чтение нескольких кнопок и логика по их состояниям
3.19	Антидребезг контактов (debounce) программным способом
3.20	Регистровое программирование vs HAL/LL-библиотеки ST
3.21	Организация кода: вынос работы с GPIO в отдельный модуль
3.22	Итоговый проект модуля 3: светомузыка на кнопках и светодиодах

Модуль 4. Таймеры, прерывания, ШИМ

Платформа: STM32 Blue Pill (bare-metal)

4.1	Концепция прерываний: что происходит при возникновении IRQ
4.2	NVIC: приоритеты прерываний и их настройка
4.3	EXTI: внешние прерывания по фронту/спаду сигнала
4.4	Практика: обработка нажатия кнопки по прерыванию вместо опроса
4.5	Антидребезг через аппаратный таймер вместо программных задержек
4.6	Обзор таймеров STM32: базовые, общего назначения, расширенные
4.7	Регистры таймера: предделитель (PSC), ARR, CNT
4.8	Настройка таймера на точный программный интервал
4.9	Прерывания от таймера: организация периодических задач
4.10	ШИМ (PWM): принцип работы и режимы таймера для генерации
4.11	Практика: плавное изменение яркости светодиода через ШИМ
4.12	Input capture: измерение длительности импульса входным сигналом
4.13	Watchdog-таймер: защита от зависания программы
4.14	Базовые режимы энергосбережения микроконтроллера (кратко)
4.15	Практика: планировщик задач round-robin на прерываниях без ОС
4.16	Итоговый проект модуля 4: ШИМ-регулятор с кнопками и watchdog

Модуль 5. Переезд на ESP32/ESP-IDF: протоколы и первый драйвер датчика

5.1	Переезд на ESP32: архитектура Xtensa/RISC-V, отличия от STM32
5.2	Настройка ESP-IDF на Ubuntu: toolchain, idf.py, menuconfig
5.3	Структура проекта ESP-IDF: CMakeLists, компоненты, sdkconfig
5.4	Первая программа на ESP-IDF: GPIO и мигание светодиодом
5.5	Введение в FreeRTOS-контекст ESP-IDF: что такое задача (кратко)
5.6	Протокол UART: физика, кадр, скорость передачи, настройка на ESP32
5.7	Практика: обмен данными по UART между ПК и ESP32
5.8	Протокол I2C: шина, адресация, master/slave, старт/стоп
5.9	Настройка I2C-мастера на ESP32 и сканирование устройств на шине
5.10	Протокол SPI: полный дуплекс, режимы CPOL/CPHA, Chip Select
5.11	Настройка SPI-мастера на ESP32 и обмен данными с периферией
5.12	Архитектура драйвера периферии: init / read / config
5.13	Датчик Холла: физический принцип, цифровой и аналоговый выход
5.14	Подключение цифрового датчика Холла к ESP32 и чтение состояния
5.15	Чтение аналогового датчика Холла через АЦП (ADC) ESP32
5.16	Калибровка и фильтрация показаний датчика Холла
5.17	Написание собственного драйвера датчика Холла: init/read/deinit
5.18	Обработка дребезга и ложных срабатываний датчика на практике
5.19	Организация драйверов в отдельные компоненты ESP-IDF
5.20	Итоговый проект модуля 5: индикация приближения магнита с настраиваемым порогом

Модуль 6. C++ в embedded

6.1	C++ в embedded: отличия от C, что можно и нельзя использовать
6.2	Классы и объекты: инкапсуляция логики драйвера
6.3	Конструкторы, деструкторы и время жизни объекта на встраиваемой системе
6.4	RAII: автоматическое управление ресурсами периферии
6.5	Почему в embedded избегают динамической памяти и исключений
6.6	Наследование и когда оно оправдано для драйверов периферии
6.7	Виртуальные функции и их цена (vtable) на микроконтроллере
6.8	Статический полиморфизм: шаблоны как альтернатива виртуальным функциям
6.9	Введение в шаблоны C++ для встраиваемых применений
6.10	Пространства имён и организация кода проекта
6.11	Перегрузка операторов: практическая польза для регистровых абстракций
6.12	Практика: обёртка GPIO-драйвера STM32 в класс GpioPin
6.13	Практика: обёртка драйвера датчика Холла ESP32 в класс HallSensor
6.14	constexpr и вычисления на этапе компиляции для embedded
6.15	Итоговый проект модуля 6: переписывание драйверов модуля 5 в C++-классы

Модуль 7. Шаговые двигатели, homing, электромагнитный захват, 3D-печать

7.1	Теория шагового двигателя: биполярный/униполярный, фазы, шаг
7.2	Микрошаг (microstepping): принцип и компромисс точность/момент
7.3	Обзор драйверов шаговых двигателей: A4988/DRV8833 vs TMC2209
7.4	Интерфейс STEP/DIR: логика управления драйвером
7.5	Ограничение тока драйвера и настройка Vref/UART TMC2209
7.6	Подключение двигателя 42SHDC3030Z-23WB-1 как 4-проводного биполярного
7.7	Программная генерация шаговых импульсов через таймер
7.8	Профили разгона/торможения для плавного движения
7.9	Ограничение скорости и защита двигателя от резких команд
7.10	Homing (поиск нулевой позиции): общие стратегии в ЧПУ и робототехнике
7.11	Homing по датчику Холла: логика и обработка граничных случаев
7.12	Концевые выключатели: программная защита от выхода за пределы
7.13	Управление электромагнитом: силовой ключ на MOSFET, снаббер-диод
7.14	Расчёт и проверка силовой части (лабораторный БП, токи, нагрев)
7.15	Интеграция: двигатель + homing + электромагнитный захват в одной программе
7.16	Введение в 3D-печать: FDM-технология, слайсер, параметры печати
7.17	Слайсинг и печать на Anycubic Kobra S1: настройка первого задания
7.18	Проектирование простого крепления под 3D-печать (базовые принципы CAD)
7.19	Печать и примерка первого крепления для двигателя/датчика
7.20	Итоговый проект модуля 7: узел захвата с homing и печатным креплением

Модуль 8. FreeRTOS

8.1	Зачем нужна RTOS: отличие от bare-metal и round-robin из модуля 4
8.2	Задачи (tasks) FreeRTOS: создание, приоритеты, стек задачи
8.3	Планировщик FreeRTOS: вытесняющая многозадачность, состояния задачи
8.4	Очереди (queues): обмен данными между задачами
8.5	Семафоры: бинарные и счётные, синхронизация задач
8.6	Мьютексы и проблема инверсии приоритетов
8.7	Группы событий (event groups) для сложной синхронизации
8.8	Программные таймеры FreeRTOS
8.9	Уведомления задач (task notifications) как лёгкая альтернатива очередям
8.10	Работа с прерываниями в контексте RTOS: ISR-safe API
8.11	Подбор размера стека задачи и диагностика переполнения
8.12	Архитектура многозадачного приложения робота: разбиение на задачи
8.13	Приоритизация задач: сенсоры, управление моторами, телеметрия
8.14	Отладка гонок данных и дедлоков в многозадачном коде
8.15	Итоговый проект модуля 8: три задачи — датчик, мотор, вывод состояния

Модуль 9. Сборка робота

Практика: пайка и механическая сборка

9.1	Общая механическая архитектура робота: обзор итоговой конструкции
9.2	Проектирование шасси и точек крепления узлов
9.3	Схема электропитания: источники, стабилизаторы, распределение по узлам
9.4	Расчёт и защита силовых цепей: предохранители, ограничение тока
9.5	Разводка проводки и разъёмов: практики аккуратного монтажа
9.6	Пайка силовых и сигнальных соединений: практика на реальных узлах
9.7	Сборка шасси и установка двигателей
9.8	Установка и подключение датчиков на корпусе
9.9	Первое включение полной электрической системы: пошаговая проверка
9.10	Интеграционное тестирование подсистем совместно
9.11	Поиск и устранение типичных сборочных неисправностей
9.12	Итоговая практика модуля 9: полностью собранный и протестированный корпус

Модуль 10. Сенсорика и одометрия

Платформа: ESP32, сенсорика

10.1	Энкодеры: инкрементальные и абсолютные, принцип считывания
10.2	Подключение энкодера к ESP32 и подсчёт импульсов через прерывания
10.3	Квадратурное декодирование для определения направления вращения
10.4	Одометрия по энкодерам: перевод импульсов в пройденное расстояние
10.5	Кинематика простого колёсного робота (дифференциальный привод)
10.6	Накопление ошибки одометрии и её источники
10.7	Введение в IMU: акселерометр и гироскоп, что они измеряют
10.8	Подключение IMU по I2C и чтение сырых данных
10.9	Калибровка IMU и устранение смещения нуля
10.10	Комплементарный фильтр: слияние данных акселерометра и гироскопа
10.11	Фильтрация шума сенсорных данных (скользящее среднее, простой low-pass)
10.12	Объединение одометрии и IMU для оценки положения робота
10.13	Итоговый проект модуля 10: вывод оценки положения робота в реальном времени

Модуль 11. Телеметрия

Платформа: ESP32, сеть

11.1	Обзор беспроводных возможностей ESP32: Wi-Fi и Bluetooth LE
11.2	Подключение ESP32 к Wi-Fi сети в режиме станции
11.3	Простой веб-сервер на ESP32 для мониторинга состояния
11.4	Протокол MQTT: концепция publish/subscribe
11.5	Настройка MQTT-клиента на ESP32 и брокера на ПК
11.6	Публикация телеметрии робота (сенсоры, состояние моторов) по MQTT
11.7	Логирование данных телеметрии для последующего анализа
11.8	Базовая панель мониторинга (dashboard) для приёма и отображения данных
11.9	Устойчивость связи: переподключение, буферизация при разрыве
11.10	Итоговый проект модуля 11: удалённый мониторинг робота в реальном времени

Модуль 12. ROS 2

12.1	Введение в ROS 2: концепция узлов, тем, сервисов
12.2	Установка и настройка окружения ROS 2 на Ubuntu
12.3	Создание первого пакета ROS 2 и простого узла
12.4	Топики: публикация и подписка на сообщения
12.5	Сервисы: запрос-ответ взаимодействие между узлами
12.6	Пользовательские типы сообщений для данных робота
12.7	Введение в micro-ROS: запуск ROS 2 узла на микроконтроллере
12.8	Настройка micro-ROS на ESP32 и связь с хостом
12.9	Публикация данных с датчиков робота (модуль 10) в топик ROS 2
12.10	Приём команд управления мотором (модули 7-8) через топик ROS 2
12.11	Введение в URDF: описание модели робота
12.12	Визуализация робота и данных в RViz
12.13	Системы координат (tf2) и их роль в работе
12.14	Запись и воспроизведение данных через ros2 bag
12.15	Организация launch-файлов для запуска всей системы
12.16	Интеграционное тестирование связки робот-ESP32-ROS2-хост
12.17	Итоговый проект модуля 12: телеуправление роботом через ROS 2

Модуль 13. Дипломный проект

Итоговая интеграция всех модулей

13.1	Формулирование темы и целей дипломного проекта
13.2	Проектирование архитектуры итоговой системы (аппаратной и программной)
13.3	Планирование этапов реализации и оценка сроков
13.4	Реализация: интеграция всех подсистем в единый проект
13.5	Тестирование системы в реальных условиях и фиксация проблем
13.6	Доработка по результатам тестирования
13.7	Подготовка технической документации проекта
13.8	Подготовка демонстрационных материалов (видео, схемы, графики)
13.9	Подготовка презентации и защитной речи
13.10	Финальная защита и разбор возможных вопросов комиссии